Sudoku Solver GenAI Comparison

1. Objective

Make the sudoku solver put all solutions in a single file

GenAI Access and Shared Prompt

How I accessed the programs: I used Gemini through its web interface and CoPilot through its desktop application

Prompt used for both GenAIs:

> //make this change: Write the solutions for each puzzle file to a single file named solutions.txt. // • For each puzzle, the solutions.txt file should include the following: // o A single line: “Puzzle: x”, where x is the puzzle file name (e.g., // puzzle1.txt). // o If no solutions are found, a single line: “No solution found.” // o If solutions are found, it should include all solutions followed by a // single line: “Solutions found: n”, where n is the number of solutions // found. // o The solutions should be in the same format as the input files: // puzzle1.txt – puzzle5.txt.

---

3. GenAI #1

GenAI Used: Gemini

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cctype>
#include <cstddef>
#include <sstream>

// ---------------------------------------------------------------------------
// Board dimensions
// ---------------------------------------------------------------------------

// Number of rows, columns, and symbols per row/column/block.
// Change this one constant to resize the board (a perfect square, 9 or less,
// because the file format uses the single digits 1-9).
constexpr int N = 9;

// Helper evaluated at compile time: smallest g (starting at "guess") with
// g*g >= n. For a perfect square n this is exactly sqrt(n).
// (Written as a single return statement so it is valid constexpr in C++11.)
```

```
constexpr int integerSqrt(int n, int guess) {
    return (guess * guess >= n) ? guess : integerSqrt(n, guess + 1);
}

// Side length of one block (3 for a 9x9 board), derived from N.
constexpr int BLOCK = integerSqrt(N, 1);

// Compile-time sanity checks: fail the build instead of misbehaving at runtime.
static_assert(BLOCK * BLOCK == N, "N must be a perfect square (4, 9, ...)");
static_assert(N >= 1 && N <= 9, "N must be 9 or less: files use the digits 1-9");

// ---------------------------------------------------------------------------
// Solving modes
// ---------------------------------------------------------------------------

// What the solver should do. Replaces the old pair of booleans
// (stopAfterFirst, countOnly), which allowed meaningless combinations.
enum class Mode {
    First,  // stop at the first solution found
    Count,  // find every solution but only count them (no grids stored)
    All     // find every solution and store each one
};

// ---------------------------------------------------------------------------
// SudokuGrid: holds one N x N board and knows how to load, check, and print it
// ---------------------------------------------------------------------------
class SudokuGrid {
private:
    int grid[N][N];          // 0 means empty, 1..N are filled cells
    std::string sourceName;  // file name the puzzle was loaded from
    std::string loadError;   // reason the last loadFromFile() call failed

    // Builds the horizontal line printed between block rows, for example
    // "------+-------+------" on a 9x9 board. The first and last segments are
    // 2*BLOCK wide; middle segments are one wider to cover the "| " dividers.
    static std::string separatorLine() {
        std::string line;
        for (int b = 0; b < BLOCK; ++b) {
            bool middle = (b > 0 && b < BLOCK - 1);
            line += std::string(2 * BLOCK + (middle ? 1 : 0), '-');
            if (b < BLOCK - 1) line += "+";
        }
        return line;
    }

public:
    // Starts with an empty board (all zeros).
    SudokuGrid() : sourceName(""), loadError("") {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c)
                grid[r][c] = 0;
    }

    void setSourceName(const std::string &name) {
```

```
        sourceName = name;
    }

    const std::string &getSourceName() const {
        return sourceName;
    }

    // Human-readable reason the last load failed (empty if it succeeded).
    const std::string &getLoadError() const {
        return loadError;
    }

    // Reads a puzzle from a text file: N non-blank lines, each holding N cells.
    // '_', '.', or '0' mean empty; '1'..'N' are givens; whitespace is ignored.
    // Returns true on success. On failure returns false and sets loadError.
    bool loadFromFile(const std::string &filename) {
        setSourceName(filename);
        loadError.clear();

        std::ifstream in(filename);
        if (!in) {
            loadError = "file is missing or cannot be opened";
            return false;
        }

        // Start from an empty board so a failed load leaves no stale data.
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                grid[r][c] = 0;
            }
        }

        // Read every line, skipping blank lines (e.g. trailing newlines).
        std::vector<std::string> rows;
        std::string line;
        while (std::getline(in, line)) {
            bool blank = true;
            for (char ch : line) {
                if (!std::isspace(static_cast<unsigned char>(ch))) {
                    blank = false;
                    break;
                }
            }
            if (!blank) rows.push_back(line);
        }

        // The file must contain exactly N usable rows.
        if (rows.size() != static_cast<std::size_t>(N)) {
            loadError = "expected " + std::to_string(N) + " rows but found " +
                        std::to_string(rows.size());
            return false;
        }

        // Parse each row symbol by symbol.
```

```
        for (int r = 0; r < N; ++r) {
            int col = 0;  // next column to fill in this row
            for (char ch : rows[r]) {
                if (std::isspace(static_cast<unsigned char>(ch))) {
                    continue;  // spaces separate symbols; ignore them
                }

                if (col >= N) {
                    loadError = "row " + std::to_string(r + 1) +
                                " has more than " + std::to_string(N) + " cells";
                    return false;
                }

                if (ch == '_' || ch == '.' || ch == '0') {
                    grid[r][col] = 0;                       // empty cell
                } else if (ch >= '1' && ch <= static_cast<char>('0' + N)) {
                    grid[r][col] = ch - '0';                // given digit
                } else {
                    loadError = std::string("invalid symbol '") + ch +
                                "' in row " + std::to_string(r + 1);
                    return false;
                }

                ++col;
            }

            if (col != N) {
                loadError = "row " + std::to_string(r + 1) + " has " +
                            std::to_string(col) + " cells instead of " +
                            std::to_string(N);
                return false;
            }
        }

        return true;
    }

    // Checks the given (non-empty) cells against each other. Every pair of
    // equal digits that share a row, column, or block is described in
    // "conflicts" (rows/columns are numbered from 1 for readability).
    // Returns true if at least one conflict exists.
    bool findConflicts(std::vector<std::string> &conflicts) const {
        conflicts.clear();

        // Treat the board as a flat list of N*N cells: index = row*N + col.
        // Comparing only later cells (b > a) reports each pair once.
        for (int a = 0; a < N * N; ++a) {
            int r1 = a / N, c1 = a % N;
            int v = grid[r1][c1];
            if (v == 0) continue;  // empty cells cannot conflict

            for (int b = a + 1; b < N * N; ++b) {
                int r2 = b / N, c2 = b % N;
                if (grid[r2][c2] != v) continue;
```

```
                bool sameRow   = (r1 == r2);
                bool sameCol   = (c1 == c2);
                bool sameBlock = (r1 / BLOCK == r2 / BLOCK) &&
                                 (c1 / BLOCK == c2 / BLOCK);
                if (!sameRow && !sameCol && !sameBlock) continue;

                // Describe which unit(s) the two cells share.
                std::string why;
                if (sameRow)   why += "same row";
                if (sameCol)   why += (why.empty() ? "" : ", ") +
std::string("same column");
                if (sameBlock) why += (why.empty() ? "" : ", ") +
std::string("same block");

                std::ostringstream msg;
                msg << "Value " << v
                    << " appears at (row " << r1 + 1 << ", col " << c1 + 1 << ")
and "
                    << "(row " << r2 + 1 << ", col " << c2 + 1 << "): " << why;
                conflicts.push_back(msg.str());
            }
        }

        return !conflicts.empty();
    }

    // Pretty console format: block dividers ("|" and "+---+") between blocks.
    void print(std::ostream &out) const {
        for (int r = 0; r < N; ++r) {
            if (r > 0 && r % BLOCK == 0) {
                out << separatorLine() << "\n";   // line between block rows
            }
            for (int c = 0; c < N; ++c) {
                if (c > 0 && c % BLOCK == 0) out << "| ";   // between block
columns
                if (grid[r][c] == 0) out << "_";
                else out << grid[r][c];
                if (c % BLOCK != BLOCK - 1) out << " ";     // space inside a
block
            }
            out << "\n";
        }
    }

    // File format: same layout as the puzzle input files
    // (N rows, symbols separated by single spaces, '_' for empty).
    void printPlain(std::ostream &out) const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                if (c > 0) out << " ";
                if (grid[r][c] == 0) out << "_";
                else out << grid[r][c];
            }
```

```
            out << "\n";
        }
    }

    // Simple accessors for a single cell.
    int get(int r, int c) const {
        return grid[r][c];
    }

    void set(int r, int c, int val) {
        grid[r][c] = val;
    }

    // True if "val" does not already appear in row r.
    bool isRowValid(int r, int val) const {
        for (int c = 0; c < N; ++c)
            if (grid[r][c] == val) return false;
        return true;
    }

    // True if "val" does not already appear in column c.
    bool isColValid(int c, int val) const {
        for (int r = 0; r < N; ++r)
            if (grid[r][c] == val) return false;
        return true;
    }

    // True if "val" does not already appear in the block containing (r, c).
    bool isBlockValid(int r, int c, int val) const {
        int br = (r / BLOCK) * BLOCK;   // top row of the block
        int bc = (c / BLOCK) * BLOCK;   // left column of the block
        for (int rr = br; rr < br + BLOCK; ++rr)
            for (int cc = bc; cc < bc + BLOCK; ++cc)
                if (grid[rr][cc] == val) return false;
        return true;
    }

    bool isCellEmpty(int r, int c) const {
        return grid[r][c] == 0;
    }

    // Finds the first empty cell in row-major order. Returns false if the
    // board is full; otherwise returns true and sets r and c.
    bool findEmptyCell(int &r, int &c) const {
        for (int i = 0; i < N; ++i)
            for (int j = 0; j < N; ++j)
                if (grid[i][j] == 0) {
                    r = i;
                    c = j;
                    return true;
                }
        return false;
    }
```

```
    // True when no empty cells remain.
    bool isComplete() const {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c)
                if (grid[r][c] == 0) return false;
        return true;
    }
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive backtracking search plus result reporting
// ---------------------------------------------------------------------------
class SudokuSolver {
private:
    SudokuGrid initial;                 // the puzzle as loaded (never modified)
    std::vector<SudokuGrid> solutions;  // stored solutions (Mode::First /
Mode::All)
    std::size_t solutionCount = 0;      // number of solutions found
    Mode mode = Mode::All;              // mode used by the most recent solve()

    // A value may go in (r, c) only if it is new to the row, column, and block.
    bool canPlace(const SudokuGrid &g, int r, int c, int val) const {
        return g.isRowValid(r, val) &&
               g.isColValid(c, val) &&
               g.isBlockValid(r, c, val);
    }

    // Recursive backtracking: fill the first empty cell with each legal value
    // and recurse; undo the placement afterwards. Returns true if at least
    // one solution was found below this point. With Mode::First the search
    // unwinds immediately after the first solution.
    bool search(SudokuGrid &current) {
        int r, c;

        // Base case: no empty cell left means the board is complete.
        if (!current.findEmptyCell(r, c)) {
            if (current.isComplete()) {
                ++solutionCount;
                if (mode != Mode::Count) {      // Count mode stores nothing
                    solutions.push_back(current);
                }
                return true;
            }
            return false;
        }

        bool foundSolution = false;
        for (int val = 1; val <= N; ++val) {
            if (canPlace(current, r, c, val)) {
                current.set(r, c, val);          // try the value
                bool found = search(current);    // explore deeper
                current.set(r, c, 0);            // backtrack

                if (found) {
```

```
                    if (mode == Mode::First) {
                        return true;            // done: only one wanted
                    }
                    foundSolution = true;       // keep looking for more
                }
            }
        }

        return foundSolution;
    }

public:
    SudokuSolver(const SudokuGrid &g) : initial(g) {}

    // Runs the search in the requested mode, replacing any earlier results.
    void solve(Mode m) {
        solutions.clear();
        solutionCount = 0;
        mode = m;
        SudokuGrid working = initial;   // search on a copy of the puzzle
        search(working);
    }

    const std::vector<SudokuGrid> &getSolutions() const {
        return solutions;
    }

    // Writes results to a solution file in the puzzle input format:
    //   - "No solution found." when nothing was found, or
    //   - each stored solution (blank line between them) followed by
    //     "Solutions found: n".
    // In Mode::Count no grids are stored, so only the count line is written.
    // Returns false if the file could not be opened.
    bool writeSolutionFile(const std::string &filename) const {
        std::ofstream out(filename);
        if (!out) return false;

        if (solutionCount == 0) {
            out << "No solution found.\n";
            return true;
        }

        for (std::size_t i = 0; i < solutions.size(); ++i) {
            if (i > 0) out << "\n";   // blank line between solutions
            solutions[i].printPlain(out);
        }
        out << "Solutions found: " << solutionCount << "\n";
        return true;
    }

    // Console report: the puzzle, then the result for the chosen mode.
    void printReport(std::ostream &out) const {
        out << "============================================================\n";
        out << "Puzzle file: " << initial.getSourceName() << "\n";
```

```
        out << "============================================================\n";
        out << "Puzzle from file:\n";
        initial.print(out);
        out << "------------------------------------------------------------\n";

        if (solutionCount == 0) {
            out << "No solution found\n";
        } else if (mode == Mode::Count) {
            out << "Total solutions found: " << solutionCount << "\n";
        } else {
            int idx = 1;
            for (const auto &sol : solutions) {
                out << "Solution " << idx++ << ":\n";
                sol.print(out);
                out << "------------------------------------------------------------
--\n";
            }
        }
        out << "============================================================\n\n";
    }
};

// ---------------------------------------------------------------------------
// Helper functions used by main()
// ---------------------------------------------------------------------------

// Builds the solution filename from the puzzle filename, keeping any folder:
//   "puzzle1.txt"        -> "solution1.txt"
//   "data/puzzle7.txt"   -> "data/solution7.txt"
//   "other.txt"          -> "solution_other.txt"  (name doesn't start with
"puzzle")
std::string makeSolutionFilename(const std::string &puzzleFile) {
    // Split into folder prefix (including the slash) and base name.
    std::size_t slash = puzzleFile.find_last_of("/\\");
    std::string dir  = (slash == std::string::npos) ? "" : puzzleFile.substr(0,
slash + 1);
    std::string base = (slash == std::string::npos) ? puzzleFile :
puzzleFile.substr(slash + 1);

    const std::string prefix = "puzzle";
    if (base.compare(0, prefix.size(), prefix) == 0) {
        return dir + "solution" + base.substr(prefix.size());
    }
    return dir + "solution_" + base;
}

// Reports a puzzle that could not be used: prints it to the console and also
// writes the same note into the solution file. "details" holds extra lines
// (for example, the list of conflicting givens).
void reportUnusablePuzzleFile(const std::string &puzzleFile,
                              const std::string &solutionFile,
                              const std::string &reason,
                              const std::vector<std::string> &details) {
    const std::string line =
```

```
"============================================================\n";

    // Console report.
    std::cout << line << "Puzzle file: " << puzzleFile << "\n" << line;
    std::cout << "Could not load puzzle: " << reason << "\n";
    for (const auto &d : details) std::cout << "  " << d << "\n";
    std::cout << line << "\n";

    // Same note in the solution file, so no stale results are left behind.
    std::ofstream out(solutionFile);
    if (!out) {
        std::cerr << "Could not write " << solutionFile << "\n";
        return;
    }
    out << "Could not load puzzle: " << reason << "\n";
    for (const auto &d : details) out << "  " << d << "\n";
}

int main() {
    // ----- Menu: choose how every puzzle will be solved -----
    int option;
    std::cout << "Choose a solving option:\n"
              << "1. Find the first solution\n"
              << "2. Count all solutions\n"
              << "3. Print all solutions\n"
              << "Enter option (1-3): ";

    if (!(std::cin >> option) || option < 1 || option > 3) {
        std::cerr << "Invalid option. Enter 1, 2, or 3.\n";
        return 1;
    }

    // Translate the menu number into a Mode once, up front.
    Mode mode = Mode::All;
    switch (option) {
        case 1: mode = Mode::First; break;
        case 2: mode = Mode::Count; break;
        case 3: mode = Mode::All;   break;
    }

    // Puzzle files to process. To add a puzzle, add its name here; its
    // solution filename is derived automatically.
    std::vector<std::string> files = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto &fname : files) {
        const std::string solutionFile = makeSolutionFilename(fname);

        // Step 1: load the puzzle. On failure, note it and move on.
```

```
        SudokuGrid grid;
        if (!grid.loadFromFile(fname)) {
            reportUnusablePuzzleFile(fname, solutionFile, grid.getLoadError(),
{});
            continue;
        }

        // Step 2: validate the givens before spending time searching.
        std::vector<std::string> conflicts;
        if (grid.findConflicts(conflicts)) {
            reportUnusablePuzzleFile(fname, solutionFile,
                                     "the given numbers conflict with each other",
                                     conflicts);
            continue;
        }

        // Step 3: solve in the chosen mode, then report and save the results.
        SudokuSolver solver(grid);
        solver.solve(mode);
        solver.printReport(std::cout);

        if (!solver.writeSolutionFile(solutionFile)) {
            std::cerr << "Could not write " << solutionFile << "\n";
        }
    }

    return 0;
}
```

4. GenAI #2

GenAI Used: CoPilot

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cctype>
#include <cstddef>
#include <sstream>

// ---------------------------------------------------------------------------
// Board dimensions
// ---------------------------------------------------------------------------
```

```
// Number of rows, columns, and symbols per row/column/block.
// Change this one constant to resize the board (a perfect square, 9 or less,
// because the file format uses the single digits 1-9).
constexpr int N = 9;

// Helper evaluated at compile time: smallest g (starting at "guess") with
// g*g >= n. For a perfect square n this is exactly sqrt(n).
// (Written as a single return statement so it is valid constexpr in C++11.)
constexpr int integerSqrt(int n, int guess) {
    return (guess * guess >= n) ? guess : integerSqrt(n, guess + 1);
}

// Side length of one block (3 for a 9x9 board), derived from N.
constexpr int BLOCK = integerSqrt(N, 1);

// Compile-time sanity checks: fail the build instead of misbehaving at runtime.
static_assert(BLOCK * BLOCK == N, "N must be a perfect square (4, 9, ...)");
static_assert(N >= 1 && N <= 9, "N must be 9 or less: files use the digits 1-9");

// ---------------------------------------------------------------------------
// Solving modes
// ---------------------------------------------------------------------------

// What the solver should do. Replaces the old pair of booleans
// (stopAfterFirst, countOnly), which allowed meaningless combinations.
enum class Mode {
    First,  // stop at the first solution found
    Count,  // find every solution and report the count on the console
    All     // find every solution and store each one
};

// ---------------------------------------------------------------------------
// SudokuGrid: holds one N x N board and knows how to load, check, and print it
// ---------------------------------------------------------------------------
class SudokuGrid {
private:
    int grid[N][N];          // 0 means empty, 1..N are filled cells
    std::string sourceName;  // file name the puzzle was loaded from
    std::string loadError;   // reason the last loadFromFile() call failed

    // Builds the horizontal line printed between block rows, for example
    // "------+-------+------" on a 9x9 board. The first and last segments are
    // 2*BLOCK wide; middle segments are one wider to cover the "| " dividers.
    static std::string separatorLine() {
        std::string line;
        for (int b = 0; b < BLOCK; ++b) {
            bool middle = (b > 0 && b < BLOCK - 1);
            line += std::string(2 * BLOCK + (middle ? 1 : 0), '-');
            if (b < BLOCK - 1) line += "+";
        }
        return line;
    }

public:
```

```
    // Starts with an empty board (all zeros).
    SudokuGrid() : sourceName(""), loadError("") {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c)
                grid[r][c] = 0;
    }

    void setSourceName(const std::string &name) {
        sourceName = name;
    }

    const std::string &getSourceName() const {
        return sourceName;
    }

    // Human-readable reason the last load failed (empty if it succeeded).
    const std::string &getLoadError() const {
        return loadError;
    }

    // Reads a puzzle from a text file: N non-blank lines, each holding N cells.
    // '_', '.', or '0' mean empty; '1'..'N' are givens; whitespace is ignored.
    // Returns true on success. On failure returns false and sets loadError.
    bool loadFromFile(const std::string &filename) {
        setSourceName(filename);
        loadError.clear();

        std::ifstream in(filename);
        if (!in) {
            loadError = "file is missing or cannot be opened";
            return false;
        }

        // Start from an empty board so a failed load leaves no stale data.
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                grid[r][c] = 0;
            }
        }

        // Read every line, skipping blank lines (e.g. trailing newlines).
        std::vector<std::string> rows;
        std::string line;
        while (std::getline(in, line)) {
            bool blank = true;
            for (char ch : line) {
                if (!std::isspace(static_cast<unsigned char>(ch))) {
                    blank = false;
                    break;
                }
            }
            if (!blank) rows.push_back(line);
        }
```

```
        // The file must contain exactly N usable rows.
        if (rows.size() != static_cast<std::size_t>(N)) {
            loadError = "expected " + std::to_string(N) + " rows but found " +
                        std::to_string(rows.size());
            return false;
        }

        // Parse each row symbol by symbol.
        for (int r = 0; r < N; ++r) {
            int col = 0;  // next column to fill in this row
            for (char ch : rows[r]) {
                if (std::isspace(static_cast<unsigned char>(ch))) {
                    continue;  // spaces separate symbols; ignore them
                }

                if (col >= N) {
                    loadError = "row " + std::to_string(r + 1) +
                                " has more than " + std::to_string(N) + " cells";
                    return false;
                }

                if (ch == '_' || ch == '.' || ch == '0') {
                    grid[r][col] = 0;                       // empty cell
                } else if (ch >= '1' && ch <= static_cast<char>('0' + N)) {
                    grid[r][col] = ch - '0';                // given digit
                } else {
                    loadError = std::string("invalid symbol '") + ch +
                                "' in row " + std::to_string(r + 1);
                    return false;
                }

                ++col;
            }

            if (col != N) {
                loadError = "row " + std::to_string(r + 1) + " has " +
                            std::to_string(col) + " cells instead of " +
                            std::to_string(N);
                return false;
            }
        }

        return true;
    }

    // Checks the given (non-empty) cells against each other. Every pair of
    // equal digits that share a row, column, or block is described in
    // "conflicts" (rows/columns are numbered from 1 for readability).
    // Returns true if at least one conflict exists.
    bool findConflicts(std::vector<std::string> &conflicts) const {
        conflicts.clear();

        // Treat the board as a flat list of N*N cells: index = row*N + col.
        // Comparing only later cells (b > a) reports each pair once.
```

```
        for (int a = 0; a < N * N; ++a) {
            int r1 = a / N, c1 = a % N;
            int v = grid[r1][c1];
            if (v == 0) continue;  // empty cells cannot conflict

            for (int b = a + 1; b < N * N; ++b) {
                int r2 = b / N, c2 = b % N;
                if (grid[r2][c2] != v) continue;

                bool sameRow   = (r1 == r2);
                bool sameCol   = (c1 == c2);
                bool sameBlock = (r1 / BLOCK == r2 / BLOCK) &&
                                 (c1 / BLOCK == c2 / BLOCK);
                if (!sameRow && !sameCol && !sameBlock) continue;

                // Describe which unit(s) the two cells share.
                std::string why;
                if (sameRow)   why += "same row";
                if (sameCol)   why += (why.empty() ? "" : ", ") +
std::string("same column");
                if (sameBlock) why += (why.empty() ? "" : ", ") +
std::string("same block");

                std::ostringstream msg;
                msg << "Value " << v
                    << " appears at (row " << r1 + 1 << ", col " << c1 + 1 << ")
and "
                    << "(row " << r2 + 1 << ", col " << c2 + 1 << "): " << why;
                conflicts.push_back(msg.str());
            }
        }

        return !conflicts.empty();
    }

    // Pretty console format: block dividers ("|" and "+---+") between blocks.
    void print(std::ostream &out) const {
        for (int r = 0; r < N; ++r) {
            if (r > 0 && r % BLOCK == 0) {
                out << separatorLine() << "\n";   // line between block rows
            }
            for (int c = 0; c < N; ++c) {
                if (c > 0 && c % BLOCK == 0) out << "| ";   // between block
columns
                if (grid[r][c] == 0) out << "_";
                else out << grid[r][c];
                if (c % BLOCK != BLOCK - 1) out << " ";     // space inside a
block
            }
            out << "\n";
        }
    }

    // File format: same layout as the puzzle input files
```

```
    // (N rows, symbols separated by single spaces, '_' for empty).
    void printPlain(std::ostream &out) const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                if (c > 0) out << " ";
                if (grid[r][c] == 0) out << "_";
                else out << grid[r][c];
            }
            out << "\n";
        }
    }

    // Simple accessors for a single cell.
    int get(int r, int c) const {
        return grid[r][c];
    }

    void set(int r, int c, int val) {
        grid[r][c] = val;
    }

    // True if "val" does not already appear in row r.
    bool isRowValid(int r, int val) const {
        for (int c = 0; c < N; ++c)
            if (grid[r][c] == val) return false;
        return true;
    }

    // True if "val" does not already appear in column c.
    bool isColValid(int c, int val) const {
        for (int r = 0; r < N; ++r)
            if (grid[r][c] == val) return false;
        return true;
    }

    // True if "val" does not already appear in the block containing (r, c).
    bool isBlockValid(int r, int c, int val) const {
        int br = (r / BLOCK) * BLOCK;   // top row of the block
        int bc = (c / BLOCK) * BLOCK;   // left column of the block
        for (int rr = br; rr < br + BLOCK; ++rr)
            for (int cc = bc; cc < bc + BLOCK; ++cc)
                if (grid[rr][cc] == val) return false;
        return true;
    }

    bool isCellEmpty(int r, int c) const {
        return grid[r][c] == 0;
    }

    // Finds the first empty cell in row-major order. Returns false if the
    // board is full; otherwise returns true and sets r and c.
    bool findEmptyCell(int &r, int &c) const {
        for (int i = 0; i < N; ++i)
            for (int j = 0; j < N; ++j)
```

```
                if (grid[i][j] == 0) {
                    r = i;
                    c = j;
                    return true;
                }
        return false;
    }

    // True when no empty cells remain.
    bool isComplete() const {
        for (int r = 0; r < N; ++r)
            for (int c = 0; c < N; ++c)
                if (grid[r][c] == 0) return false;
        return true;
    }
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive backtracking search plus result reporting
// ---------------------------------------------------------------------------
class SudokuSolver {
private:
    SudokuGrid initial;                 // the puzzle as loaded (never modified)
    std::vector<SudokuGrid> solutions;  // solutions written to the output file
    std::size_t solutionCount = 0;      // number of solutions found
    Mode mode = Mode::All;              // mode used by the most recent solve()

    // A value may go in (r, c) only if it is new to the row, column, and block.
    bool canPlace(const SudokuGrid &g, int r, int c, int val) const {
        return g.isRowValid(r, val) &&
               g.isColValid(c, val) &&
               g.isBlockValid(r, c, val);
    }

    // Recursive backtracking: fill the first empty cell with each legal value
    // and recurse; undo the placement afterwards. Returns true if at least
    // one solution was found below this point. With Mode::First the search
    // unwinds immediately after the first solution.
    bool search(SudokuGrid &current) {
        int r, c;

        // Base case: no empty cell left means the board is complete.
        if (!current.findEmptyCell(r, c)) {
            if (current.isComplete()) {
                ++solutionCount;
                solutions.push_back(current);
                return true;
            }
            return false;
        }

        bool foundSolution = false;
        for (int val = 1; val <= N; ++val) {
            if (canPlace(current, r, c, val)) {
```

```
                current.set(r, c, val);          // try the value
                bool found = search(current);    // explore deeper
                current.set(r, c, 0);            // backtrack

                if (found) {
                    if (mode == Mode::First) {
                        return true;             // done: only one wanted
                    }
                    foundSolution = true;        // keep looking for more
                }
            }
        }

        return foundSolution;
    }

public:
    SudokuSolver(const SudokuGrid &g) : initial(g) {}

    // Runs the search in the requested mode, replacing any earlier results.
    void solve(Mode m) {
        solutions.clear();
        solutionCount = 0;
        mode = m;
        SudokuGrid working = initial;   // search on a copy of the puzzle
        search(working);
    }

    const std::vector<SudokuGrid> &getSolutions() const {
        return solutions;
    }

    // Appends results to the shared solutions file in the puzzle input format:
    //   - "No solution found." when nothing was found, or
    //   - each stored solution (blank line between them) followed by
    //     "Solutions found: n".
    // Returns false if the file could not be opened.
    bool appendSolutionFile(const std::string &filename) const {
        std::ofstream out(filename, std::ios::app);
        if (!out) return false;

        out << "Puzzle: " << initial.getSourceName() << "\n";
        if (solutionCount == 0) {
            out << "No solution found.\n";
            return true;
        }

        for (std::size_t i = 0; i < solutions.size(); ++i) {
            if (i > 0) out << "\n";   // blank line between solutions
            solutions[i].printPlain(out);
        }
        out << "Solutions found: " << solutionCount << "\n";
        return true;
    }
```

```
    // Console report: the puzzle, then the result for the chosen mode.
    void printReport(std::ostream &out) const {
        out << "============================================================\n";
        out << "Puzzle file: " << initial.getSourceName() << "\n";
        out << "============================================================\n";
        out << "Puzzle from file:\n";
        initial.print(out);
        out << "------------------------------------------------------------\n";

        if (solutionCount == 0) {
            out << "No solution found\n";
        } else if (mode == Mode::Count) {
            out << "Total solutions found: " << solutionCount << "\n";
        } else {
            int idx = 1;
            for (const auto &sol : solutions) {
                out << "Solution " << idx++ << ":\n";
                sol.print(out);
                out << "------------------------------------------------------------
--\n";
            }
        }
        out << "============================================================\n\n";
    }
};

// ---------------------------------------------------------------------------
// Helper functions used by main()
// ---------------------------------------------------------------------------

// Reports a puzzle that could not be used: prints it to the console and also
// writes a no-solution entry into the shared results file. "details" holds
// extra lines (for example, the list of conflicting givens).
void reportUnusablePuzzleFile(const std::string &puzzleFile,
                              const std::string &resultsFile,
                              const std::string &reason,
                              const std::vector<std::string> &details) {
    const std::string line =
"============================================================\n";

    // Console report.
    std::cout << line << "Puzzle file: " << puzzleFile << "\n" << line;
    std::cout << "Could not load puzzle: " << reason << "\n";
    for (const auto &d : details) std::cout << "  " << d << "\n";
    std::cout << line << "\n";

    // Record this puzzle in the shared results file.
    std::ofstream out(resultsFile, std::ios::app);
    if (!out) {
        std::cerr << "Could not write " << resultsFile << "\n";
        return;
    }
    out << "Puzzle: " << puzzleFile << "\n";
```

```
    out << "No solution found.\n";
}

int main() {
    // ----- Menu: choose how every puzzle will be solved -----
    int option;
    std::cout << "Choose a solving option:\n"
              << "1. Find the first solution\n"
              << "2. Count all solutions\n"
              << "3. Print all solutions\n"
              << "Enter option (1-3): ";

    if (!(std::cin >> option) || option < 1 || option > 3) {
        std::cerr << "Invalid option. Enter 1, 2, or 3.\n";
        return 1;
    }

    // Translate the menu number into a Mode once, up front.
    Mode mode = Mode::All;
    switch (option) {
        case 1: mode = Mode::First; break;
        case 2: mode = Mode::Count; break;
        case 3: mode = Mode::All;   break;
    }

    // Puzzle files to process. Results for every puzzle go into solutions.txt.
    std::vector<std::string> files = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    const std::string resultsFile = "solutions.txt";
    std::ofstream clearFile(resultsFile);
    if (!clearFile) {
        std::cerr << "Could not open " << resultsFile << " for writing.\n";
        return 1;
    }
    clearFile.close();

    for (const auto &fname : files) {
        // Step 1: load the puzzle. On failure, note it and move on.
        SudokuGrid grid;
        if (!grid.loadFromFile(fname)) {
            reportUnusablePuzzleFile(fname, resultsFile, grid.getLoadError(), {});
            continue;
        }

        // Step 2: validate the givens before spending time searching.
        std::vector<std::string> conflicts;
        if (grid.findConflicts(conflicts)) {
            reportUnusablePuzzleFile(fname, resultsFile,
```

```
                                    "the given numbers conflict with each other",
                                    conflicts);
            continue;
        }

        // Step 3: solve in the chosen mode, then report and save the results.
        SudokuSolver solver(grid);
        solver.solve(mode);
        solver.printReport(std::cout);

        if (!solver.appendSolutionFile(resultsFile)) {
            std::cerr << "Could not write results for " << fname
                      << " to " << resultsFile << "\n";
        }
    }

    return 0;
}
```

Correctness

- Both programs are functionally correct for standard 9x9 Sudoku solving using backtracking.
- They parse board input rows correctly, accept `_`, `.`, and `0` as empty cells, ignore whitespace, and reject invalid characters or malformed row lengths.
- Each solver validates the givens before searching by checking for duplicate values in the same row, column, or 3x3 block.
- The recursive search correctly tries legal values, writes the value, recurses, and then backtracks by clearing the cell.
- Both versions handle the three solving modes (`First`, `Count`, and `All`) consistently.
- The main correctness caveat is in reporting: both programs treat an invalid or conflicting puzzle as "No solution found." in the output file. That is acceptable for assignment output formatting, but it does not clearly distinguish between a truly unsolvable board and a malformed/invalid input board.

Time Complexity

- Both implementations use depth-first backtracking, so the dominant worst-case running time is exponential in the number of empty cells.
- If k cells are blank, the loose upper bound is O(9^k). For a fixed 9x9 Sudoku, this is effectively O(9^(N^2)) in the absolute worst case.
- Each recursion step also checks row, column, and block validity, which adds a constant-factor scan over at most 9 cells per row/column/block, so the per-node work remains small compared with the branching factor.
- The practical runtime is therefore dominated by the number of recursive branches explored, which is why these solvers are exponential in the worst case.
- `Mode::Count` and `Mode::All` can be slower than `Mode::First` because they continue searching after the first valid board instead of stopping early.

Space Complexity

- The board itself is a fixed-size 9x9 grid, so storage for the puzzle is O(N^2), which is effectively O(1) for this assignment.
- The recursive call stack grows to the number of empty cells in the current path, so auxiliary search space is O(k) in the worst case.
- The largest memory cost is in `Mode::All`: storing all discovered solutions requires O(solutions * N^2) space, which can become very large because the number of valid Sudoku solutions is exponential.
- In both programs, the asymptotic memory profile is therefore the same: search stack plus board storage, with additional exponential memory if all solutions are retained.

Readability and Maintainability

- Both programs are readable because they separate responsibilities into `SudokuGrid`, `SudokuSolver`, and a `Mode` enum.
- The naming is mostly clear: `findConflicts`, `canPlace`, `findEmptyCell`, and `printReport` all express intent well.
- `Gemini.cpp` includes more explanatory comments and is easier for a reader to follow step by step, but it also has a few redundant or awkward checks.
- `CoPilot.cpp` is the cleaner implementation overall:
  - helper methods such as `findEmptyCell`, `isCellEmpty`, and `isBlockValid` make the search logic easier to follow
  - the structure is slightly more direct and consistent
  - the comments are still informative without being excessive
- Both implementations are appropriate for assignment-level work, but `CoPilot.cpp` is a little more polished and easier to maintain.

Overall Assessment

- Correctness: both are correct for valid Sudoku inputs and standard backtracking logic.
- Complexity: both are expectedly exponential in the number of empty cells, which is unavoidable for exact Sudoku solving.
- Readability: both are understandable, but `CoPilot.cpp` is cleaner and slightly easier to maintain.
- Recommendation: If the task is to choose the better implementation, `CoPilot.cpp` is the stronger version because it is a little more structured and less redundant while preserving the same algorithmic behavior.

My Implementation:

goals:

-make the program handle errors more cleanly -eliminate the user input at the beginning because that is not consistent with the prompt -make all solutions go into a single `solutions.txt`